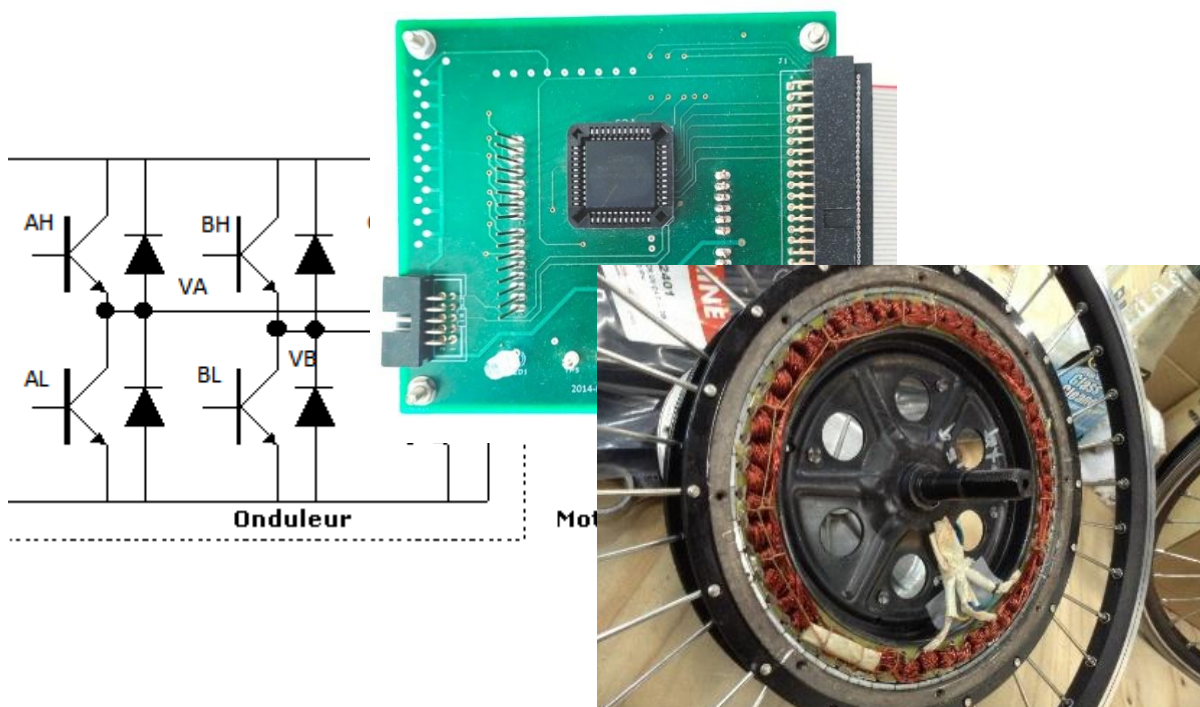


RAPPORT SAE Roue

**2024-2025
BUT2 S4 GEII
Université Côte d'Azur**



Groupe 2 :

**CIAMPI Corentin
FRANCES Quentin**

Enseignants de projet :

**LAURENT Philippe
BARAT Christian**

Table des matières

Introduction	3
I. Présentation du projet	4
II. Le PLD.....	6
a. La roue et son moteur brushless.....	6
b. Le PWM	7
c. Filtre HALL	9
d. Collecteur Numérique	10
III. Le code Mbed.....	12
a. Sélection rapport cyclique	12
b. Visualisation des états du PLD.....	12
c. Calcul de la vitesse.....	14
d. Poignée de gaz	14
e. Fichier d'enregistrement	15
f. Bridage et freinage.....	16
g. Compteurs kilométriques	17
h. Capteurs.....	17
i. Liaison avec le serveur Web	19
IV. NodeRED	20
Conclusion.....	23
Table des figures.....	24

Introduction

Durant le semestre 4 de la formation à l'IUT dans le département GEII (Génie Electrique et Informatique Industrielle), nous avons participé à un nouveau module de SAE (situation d'apprentissage et d'évaluation).

Lors de cette SAE, nous avons réalisé l'étude et le fonctionnement d'une roue électrique à partir d'un cahier des charges qui sera détaillé plus tard.

Ce projet est réalisé en groupe de 2 étudiants, groupe ici composé de CIAMPI Corentin et FRANCES Quentin. Le projet est séparé en plusieurs étapes : tout d'abord réaliser un programme en VHDL sur FPGA pour contrôler la partie électrotechnique de la roue, puis une partie Mbed en C++ pour créer un tableau de bord pour l'utilisateur et enfin une partie Web pour créer une interface utilisateur sur un objet connecté tel qu'un smartphone.

Vous trouverez tout au long de ce rapport toutes les différentes étapes par lesquelles nous sommes passés pour réaliser le travail qui nous a été demandé.

I. Présentation du projet

Comme il a été dit plus haut, nous allons réaliser une étude de la commande et du fonctionnement d'un vélo à assistance électrique. Nous aurons donc à notre disposition, une roue soit de vélo soit de scooter, dans notre cas il s'agira d'une roue de vélo. Celle-ci sera équipée d'un moteur brushless 48V de 250W, elle disposera de 3 capteurs de position du rotor, ainsi que de 8 paires de pôles.

Pour cela, le projet sera découpé en différentes parties.

En premier lieu, il y aura la programmation en VHDL d'un PLD Altera, qui sera la commande du collecteur électronique du moteur de VAE. Il permettra de faire tourner la roue, mais sans pouvoir la contrôler.

Ensuite, nous aurons la programmation en C++ d'un microcontrôleur MBED (LPC1768), qui agira comme un superviseur. Il nous permettra de contrôler la roue, notamment en pouvant choisir la vitesse qu'on souhaite lui donner, ou encore brider l'assistance électrique à une certaine vitesse.

Pour finir, nous créerons une interface, à partir de laquelle, les utilisateurs pourront voir des informations sur leur vélo en temps réel. Pour cela nous mettrons en place un serveur web ainsi qu'une application mobile, relayant des données comme la vitesse de la roue ou bien la température.



Figure 1 : VAE

Pour cette SAE, le cahier des charges est très ambitieux, étant donné les possibilités que nous offre ce projet. Nous pouvons les résumer avec l'aide du synoptique ci-dessous :

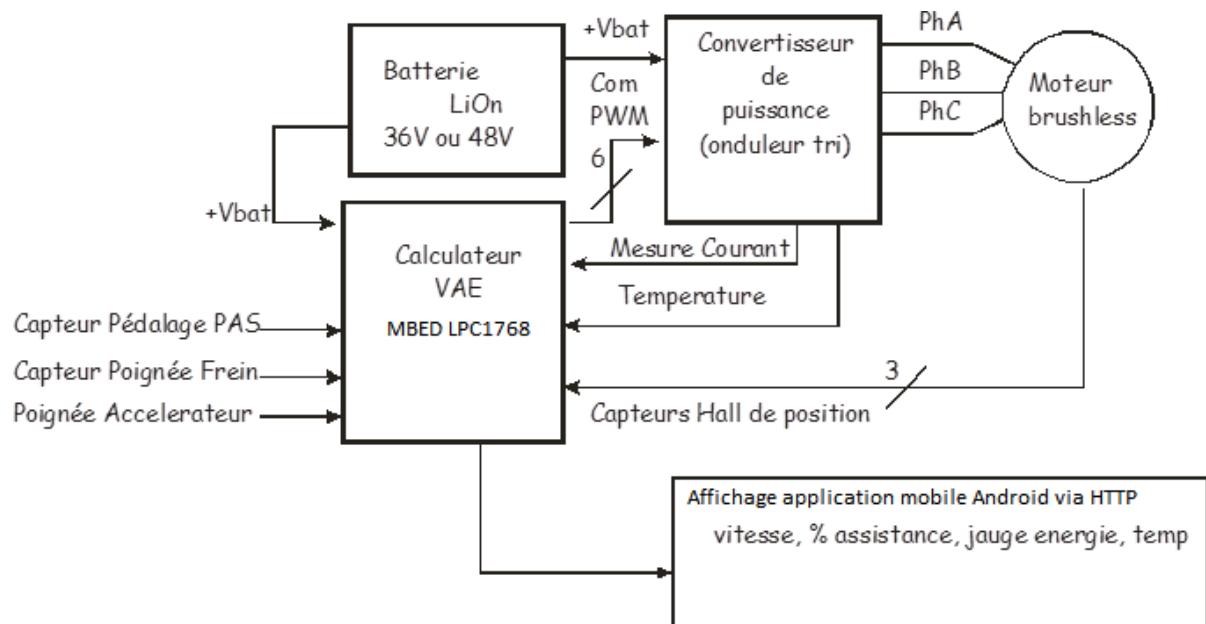


Figure 2 : Synoptique du projet

Comme on peut le voir, les principaux objectifs y sont indiqués, tels que l'affichage avec une application mobile, le convertisseur de puissance qui correspond au PLD, le calculateur VAE qui est la partie MBED. On peut voir aussi d'autres éléments moins marquants, comme la mesure du courant, de la température, et de la vitesse à partir des capteurs à effet Hall de positions (capteurs de position du rotor).

II. Le PLD

Pour commencer, nous allons voir le PLD Altera, qui nous permettra de faire tourner la roue. Afin de réussir cela, il faut déjà comprendre comment fonctionne la roue et son moteur brushless.

a. La roue et son moteur brushless

La partie puissance est assurée par un onduleur triphasé préfabriqué sur lequel nous avons extrait et dirigé les 6 commandes de transistor des bras d'onduleur vers le PLD ALTERA à savoir AH/AL, BH/BL, CH/CL (c'est le PLD qui assurera donc la commande des 3 bras d'onduleur) à partir de la connaissance de la position instantanée du rotor (inducteur) du moteur.

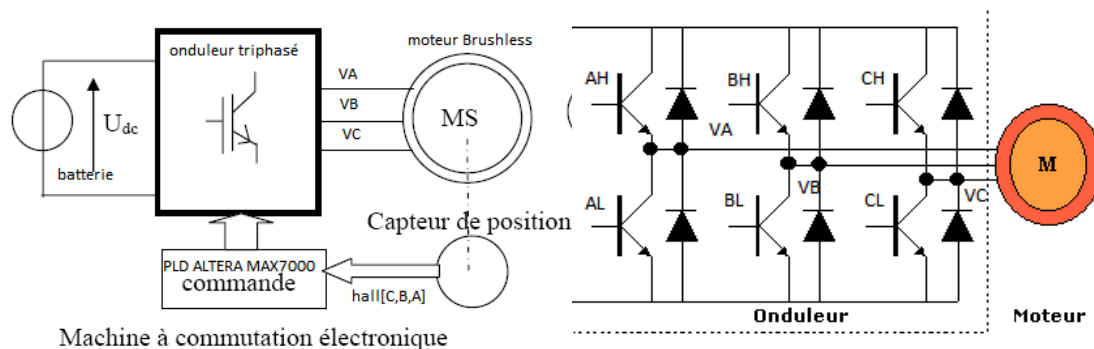


Figure 3 : Onduleur

A partir de là il élabore les 3 consignes de tension VA,VB,VC en fonction de la position actuelle du rotor (capteurs Hall[C,B,A]) et de la consigne de vitesse que désire l'utilisateur (Référence PWM au format 8 bits // fournis par le uC MBED au PLD ALTERA).

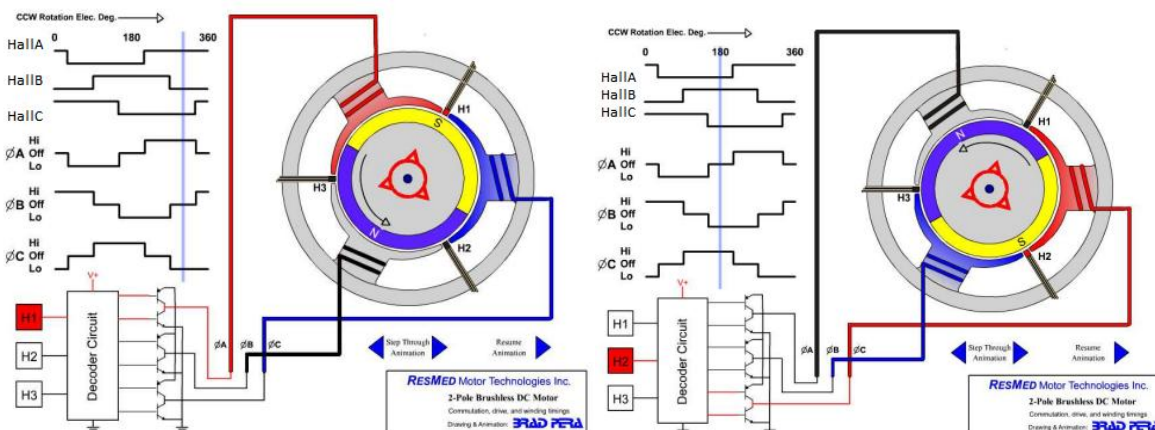


Figure 4 : Position 1 et 2

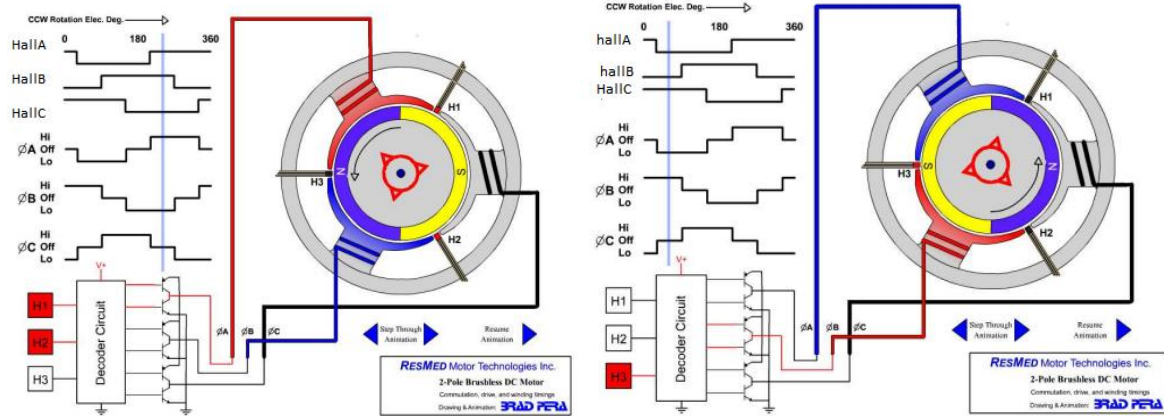


Figure 5 : Position 3 et 4

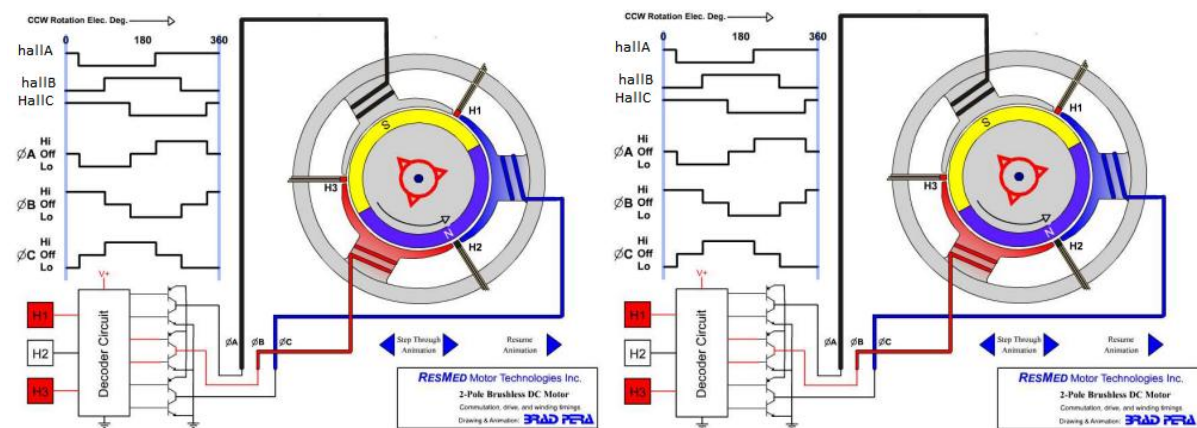


Figure 6 : Position 5 et 6

La commande des 6 transistors de l'onduleur doit être « SYNCHRON » de la position angulaire instantanée du rotor (c'est ce que l'on appelle l'autopilotage du brushless). Cette fonction est en fait l'équivalent d'un collecteur numérique.

Nous devons donc réaliser ce collecteur numérique, sur Quartus, à l'aide d'un code en VHDL.

b. Le PWM

Afin d'arriver à obtenir ce résultat, différentes étapes étaient nécessaires. Pour commencer, nous avons réalisé un bloc que nous avons nommé "PWM", celui-ci nous permet dans un premier temps de convertir l'horloge en 2,5MHz fourni par la carte Altera en une horloge de 10kHz, ainsi que de créer une PWM en fonction d'une valeur choisie.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  ENTITY PWM IS
5  PORT (
6      H_2_5_MHZ, VALID_PWM : IN STD_LOGIC;
7      REF_PWM : IN INTEGER RANGE 0 to 255; --entree rapport cyclique
8      H_10KHZ : BUFFER STD_LOGIC;
9      PWM_OUT : OUT STD_LOGIC
10 );
11 END PWM ;
12
13 ARCHITECTURE archi OF PWM IS
14     SIGNAL CPT_CLK : INTEGER RANGE 0 TO 255;
15 BEGIN
16     PROCESS(H_2_5_MHZ)
17     BEGIN
18         IF (H_2_5_MHZ'EVENT AND H_2_5_MHZ='1') THEN
19             IF CPT_CLK = 255 THEN
20                 CPT_CLK <= 0;
21                 H_10KHZ <= '1';
22             ELSE
23                 CPT_CLK <= CPT_CLK + 1;
24                 H_10KHZ <= '0';
25             END IF;
26         END IF;
27     END IF;
28 END PROCESS;
29
30 PWM_OUT <= '1' WHEN ((REF_PWM > CPT_CLK) AND VALID_PWM = '1') ELSE '0';
31
32 END archi;
33

```

Figure 7 : VHDL : PWM

Premièrement, nous créons un “PROCESS” dont l’évènement est l’horloge de 2.5MHz. A chaque coup d’horloge, elle incrémentera de un la variable “CPT_CLK”. Lorsque le compteur aura atteint la valeur ‘255’, on remettra alors le compteur à ‘0’, et on passera la variable de sortie binaire “H_10kHz” à ‘1’. Nous avons choisi un compteur allant jusqu’à ‘255’, car nous voulions une horloge de 10kHz, il fallait donc diviser 2.5MHz par 255 pour l’avoir. Dans la zone concurrente, nous avons créé une PWM qui est active seulement lorsque le compteur est inférieur à “REF_PWM”, le cahier des charges nous imposait également d’activer la PWM uniquement lorsque “VALID_PWM” était à ‘1’. “REF_PWM” nous permet donc de contrôler la vitesse de la roue en affectant son rapport cyclique. Une fois ce code fait, on crée le Bloc, afin de pouvoir l’ajouter au .bdf.

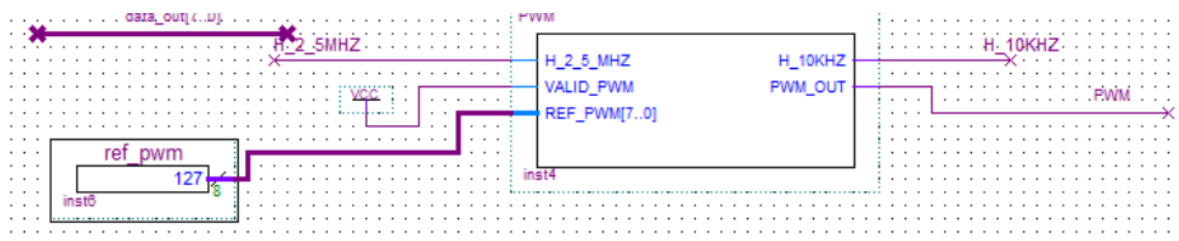


Figure 8 : Bloc PWM

c. Filtre HALL

Ensuite, nous avons fait un Bloc que nous avons appelé “filtre_hall”, celui-ci nous permettra de récupérer l’état des capteurs Hall ainsi que de filtrer lors des changements de position qui ne sont pas des états stables.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  ENTITY filtre_hall IS
5  PORT (
6      hall_A, hall_B, hall_C, H_10khz : IN STD_LOGIC;
7      valid_hall : OUT STD_LOGIC;
8      hall_filtre : OUT STD_LOGIC_VECTOR (2 downto 0)
9  );
10 END filtre_hall;
11
12 ARCHITECTURE archi OF filtre_hall IS
13     SIGNAL N_3, N_2, N : STD_LOGIC_VECTOR (2 downto 0);
14 BEGIN
15     PROCESS (H_10khz)
16     BEGIN
17         IF (H_10khz'EVENT AND H_10khz = '1') THEN
18             N_3 <= N_2;
19             N_2 <= N;
20             N <= hall_C & hall_B & hall_A;
21             IF ((N_3 = N_2) AND (N_3 = N) AND (N_2 = N)) THEN
22                 valid_hall <= '1';
23                 hall_filtre <= N;
24             ELSE
25                 valid_hall <= '0';
26             END IF;
27         END IF;
28     END PROCESS;
29
30 END archi;

```

Figure 9 : VHDL filtre HALL

A présent, nous avons une horloge de 10kHz qui sera l’évènement de notre “PROCESS”. A chaque coup d’horloge, nous enregistrerons la valeur de “N_2” dans “N_3”, celle de “N” dans “N_2”, et “N” lui récupèrera la valeur des différents capteurs Hall (“hall_C”, “hall_B”, “hall_A”), c’est un registre à décalage. Grâce à ce registre à décalage, nous pourrions comparer “N”, “N_2” et “N_3”, afin d’avoir “valid_hall” qui vaut ‘1’ uniquement lorsque les trois sont égaux. Cela permettra d’éviter les états instables et de récupérer les valeurs des capteurs Halls.

Nous pourrions alors créer le Bloc “filtre_hall”, et l’ajouter au .bdf.

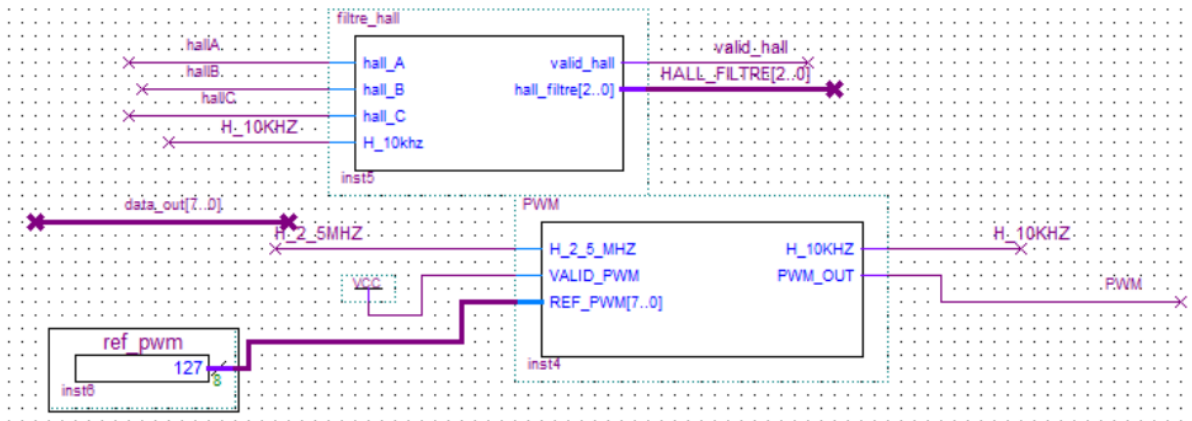


Figure 10 : bloc filtre HALL

d. Collecteur Numérique

Pour finir le PLD, nous avons réalisé un Bloc que nous avons appelé “collecteur_num”, celui-ci permettra d’alimenter Hall A, Hall B ou Hall C selon l’état dans lequel le moteur se trouve.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  ENTITY collecteur_num IS
5  PORT(
6    H_10KHZ, VALID_HALL, DIR, PWM_IN : IN STD_LOGIC ;
7    HALL_FILTRE : IN INTEGER range 0 to 7;
8    PWM_AH, PWM_AL, PWM_BH, PWM_BL, PWM_CH, PWM_CL : OUT STD_LOGIC
9  );
10 END collecteur_num ;
11
12 ARCHITECTURE archi OF collecteur_num IS
13   SIGNAL Command : STD_LOGIC_VECTOR (5 downto 0);
14 BEGIN
15   PROCESS(H_10KHZ)
16   BEGIN
17     IF(H_10KHZ'EVENT AND H_10KHZ='1') THEN
18       IF(VALID_HALL='1')THEN
19         CASE HALL_FILTRE IS
20           WHEN 0 => Command <= "000000";
21           WHEN 1 => Command <= "100001"; --AH et CL
22           WHEN 2 => Command <= "011000"; --AL et BH
23           WHEN 3 => Command <= "001001"; --CL et BH
24           WHEN 4 => Command <= "000110"; --BL et CH
25           WHEN 5 => Command <= "100100"; --AH et BL
26           WHEN 6 => Command <= "010010"; --AL et CH
27           WHEN 7 => Command <= "000000";
28         END CASE;
29       END IF;
30     END IF;
31   END PROCESS;
32
33   PWM_CL <= Command(0);
34   PWM_CH <= Command(1) AND PWM_IN;
35   PWM_BL <= Command(2);
36   PWM_BH <= Command(3) AND PWM_IN;
37   PWM_AL <= Command(4);
38   PWM_AH <= Command(5) AND PWM_IN;
39
40 END archi;
41
42
```

Figure 11 : VHDL Collecteur Numérique

Voici le code VHDL que nous avons réalisé pour le collecteur numérique. Comme nous pouvons le voir, l'évènement du "PROCESS" est l'horloge de 10kHz. A chaque coup d'horloge on vérifie que "VALID_HALL" vaut '1', si c'est le cas, on rentre dans un "CASE" avec comme variable de contrôle "HALL_FILTRE". "HALL_FILTRE" est une variable codée sur 3 BITS. Le bit de poids faible correspond à Hall A, celui d'après à Hall B enfin le bit de poids fort à Hall C. Cette variable correspond à un chiffre allant de '0' pour tous les Halls inactifs, à '7' pour tous les Halls actifs. On a donc ensuite attribué une valeur à "Command" sur 6 BITS selon la valeur de "HALL_FILTRE". Par exemple, pour une valeur de "HALL_FILTRE" égale à '6' on a, Hall A bas et Hall C haut qui sont alimentés. Pour finir, afin d'attribuer leur alimentation aux différents secteurs, on va faire correspondre par exemple le bit de poids faible de "Command" au secteur CL. On n'aura alors plus qu'à faire la commande suivante :

```
PWM_CL <= Command(0);
```

Celle-ci nous permettra d'attribuer à la variable "PWM_CL" la valeur du bit '0' de "Command". On répètera alors la même manipulation pour tous les autres secteurs, et pour les cas où "HALL_FILTRE" vaut '0' ou '7', on mettra "Command" à '0'.

Pour finir, il faudra créer le bloc "collecteur_numérique", puis l'ajouter au .bdf.

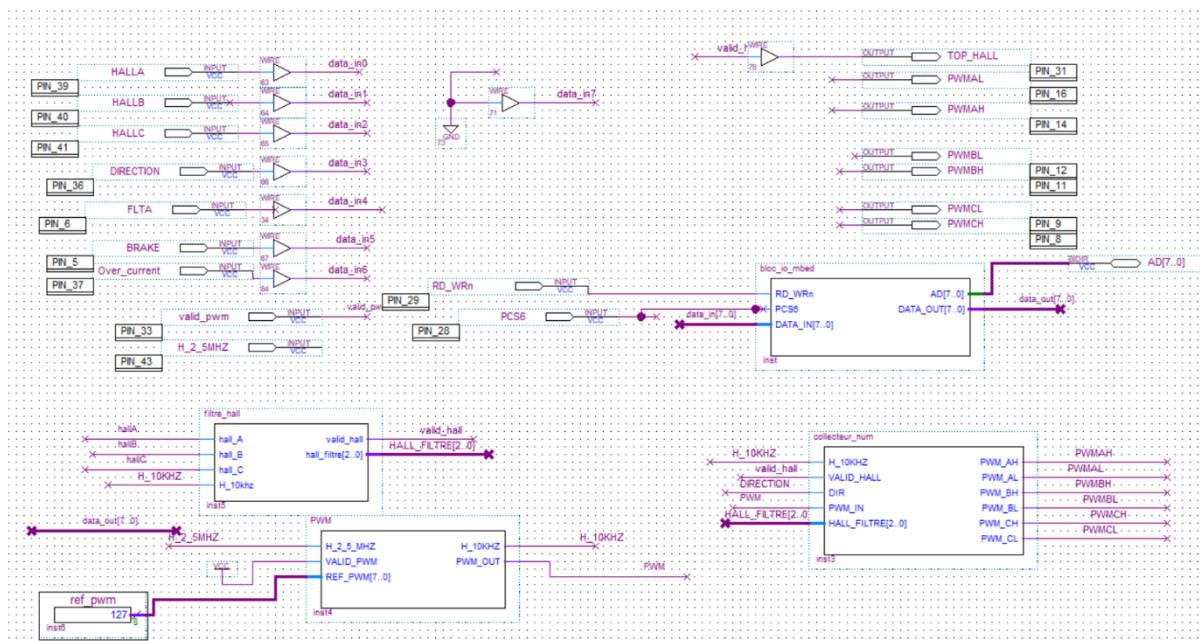


Figure 12 : BDF entier

Voilà le programme global qu'on obtient, celui-ci permettra de faire tourner la roue en autopilotage.

III. Le code Mbed

A présent, il nous faut créer un code Mbed en C++ pour créer un tableau de bord, dans un premier temps dans une fenêtre de debug, puis dans une interface graphique.

A noter qu'au démarrage du programme, une initialisation se fait afin de mettre la roue à l'arrêt.

a. Sélection rapport cyclique

Dans un premier temps, nous développons une méthode pour choisir et fixer un rapport cyclique entre 0 et 255 via le terminal de commande. Pour ce faire, nous ajoutons cette options au menu :

```
pc.printf("\n\n\rBIENVENUE DANS LA FABULEUSE GRANDE ROUE (pas roux) !");  
pc.printf("\n\rVeuillez choisir votre choix : ");  
pc.printf("\n\r1 : Saisir rapport cyclique (entre 0 et 255)");
```

Figure 13 : IHM1

Lorsque l'utilisateur rentrera le chiffre 1, il arrivera dans un menu qui va lui permettre de rentrer une valeur entre 0 et 255. Une fois cela fait, la valeur est écrite dans un objet du nom de *MyPLD*. Cet objet est lié à certaines broches du PLD, et la fonction *.write()* associée à cet objet permet notamment de rentrer le rapport cyclique voulu dans le PLD. On vérifie également que la valeur saisie est bien comprise entre 0 et 255.

```
case '1':  
    mode_auto = false;  
    pc.printf("\n\rEntrez la valeur du rapport cyclique :");  
    pc scanf("%d", &duty_cycle_manu);  
    if(duty_cycle_manu > 255) duty_cycle_manu = 255;  
    else if(duty_cycle_manu < 0) duty_cycle_manu = 0;  
    VALID_PWM.write(1);  
    MyPLD.write(duty_cycle_manu);  
    pc.printf("\n\rPWM mis a jour a : %d", duty_cycle_manu);  
    break;
```

Figure 14 : menu rapport cyclique

b. Visualisation des états du PLD

Le PLD, sur certaines broches lisibles grâce à la méthode *MyPLD.read()*, renvoie différentes informations comme la direction de la roue (avant/arrière), s'il y a un courant trop élevé ou non, s'il y a une erreur ou non et si le frein est engagé. Lorsque l'on appelle cette méthode voici ce que l'on obtient :

Over curent	Frein (actif à 0)	FLTA	Direction	HALL_C	HALL_B	HALL_A
Bit6(MSB)	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0 (LSB)

Prenons un exemple :

Bit6(MSB)	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0 (LSB)
0	1	0	1	1	0	1
Pas d'overcurrent	Frein On	Pas d'erreur	Avant	Secteur	5	actif

Comme on le voit, on a donc un grand nombre d'informations dans une seule chaîne de 1 et de 0. Pour séparer chaque information et les placer dans des variables, il va falloir effectuer un masquage. Cela consiste à décaler de x bits cette chaîne pour ensuite faire un ET bit à bit avec la valeur 0x01 (soit 1 en décimal) pour extraire uniquement la dernière information. Par exemple, pour récupérer le Bit3 voici comment on fait :

Chaîne initiale : 0101101 $\xrightarrow{\text{décalage de 3 } (\gg 3)}$ Chaîne décalée : 0000101
ET bit à bit : & 0000001

Et on récupère donc le dernier bit de la chaîne, ici le dernier 1 qui représente la direction. On peut donc le stocker dans une variable afin de l'exploiter. On fait de même avec toutes les informations, comme visible ci-dessous :

```
readPLD = MyPLD.read();
HALL = 0x07 & readPLD;
direction = 0x01 & (readPLD >> 3);
over_current = 0x01 & (readPLD >> 6);
FLTA = 0x01 & (readPLD >> 4);
BRAKE = 0x01 & (readPLD >> 5);
```

Figure 15 : Masquage du MyPLD.read()

On peut ensuite les afficher, dans le menu numéro 2 :

```
pc.printf("\n\rValeur secteur %d", HALL);
pc.printf("\n\rDirection : ");
if(direction == 1)pc.printf("marche avant");
else pc.printf("marche arriere");
if(over_current == 0)pc.printf("\n\r! OVER CURRENT !");
else pc.printf("\n\rCourant normal.");
if(FLTA == 1) pc.printf("\n\rERROR : FLTA");
else pc.printf("\n\rPas d'erreur");
if(BRAKE == 0) pc.printf("\n\rFreinage");
else pc.printf("\n\rFrein relache");
```

Figure 16 : Affichage des données du PLD

c. Calcul de la vitesse

Pour calculer la vitesse de la roue, il faut prendre en compte différents paramètres comme la taille de la roue et le nombre de paire de pôles. De plus il faut pouvoir compter à chaque fois que l'on change de secteur d'alimentation. Pour ce faire, on utilise une broche interruptIn, qui incrémentera un compteur à chaque front montant. Toutes ces données vont nous permettre de calculer une distance. Il ne reste plus qu'à déterminer une période d'échantillonnage, pour diviser cette distance par ce temps pour avoir la vitesse en mètre par secondes. Voici donc la formule :

$$V = \frac{cptHALLTOP * 2 * \pi * Rayon_{roue}}{T * 6 * N_{pôles}}$$

C'est donc ce qui est réalisé dans un ticker, avec une période d'échantillonnage de 100ms.

```
vitesse = (cptHALLTop*2*PI*RADIUS)/(DELTA_T*6*NBPOLES);
vitesse_km_h = vitesse * 3.6;
cptHALLTop = 0;
```

Figure 17 : Calcul vitesse

d. Poignée de gaz

Nous pouvons à présent passer à la gestion de la poignée de gaz. A noter que lorsqu'elle sera terminée, 2 modes seront disponibles pour la roue, un mode automatique dans lequel c'est la poignée de gaz qui contrôle la vitesse de la roue et un mode manuel où c'est l'utilisateur, via le terminal, qui fixe une vitesse (une valeur entre 0 et 255).

La poignée est connectée à une entrée analogique du microcontrôleur, qui renvoie donc une valeur entre 0 et 1. Cependant la poignée, elle, n'envoie pas une valeur entre 0 et 1 mais par exemple entre 0,243 et 0,792. Il faut donc faire quelques calculs, une simple

règle de 3, pour « transformer » ces valeurs en 0 et 1, puis entre 0 et 255, comme réalisé ci-dessous :

```
val_gaz = V_GAZ.read();
DC_gaz = (val_gaz - min_gaz) / (max_gaz - min_gaz);
if(DC_gaz < 0) DC_gaz = 0;
else if(DC_gaz > 1) DC_gaz = 1;
DC_gaz = DC_gaz * 255;
```

Figure 18 : Calcul poignée de gaz

Chaque poignée étant différente, nous avons mis en place un menu de calibrage pour cette dernière, qui va venir définir le *min_gaz* et le *max_gaz* de la poignée.

```
MyPLD.write(0);
pc.printf("\n\r-----Calibrage poignee-----");
pc.printf("\n\rMettez les gaz a 0 et appuyez sur une touche");
pc scanf("%d", &a);
min_gaz = V_GAZ.read();
pc.printf("\n\rMettez a present les gaz a fond et appuyez sur une touche");
pc scanf("%d", &a);
max_gaz = V_GAZ.read();
if(max_gaz > min_gaz){
    gaz_calibre = true;
    pc.printf("\n\r-----Poignee calibree-----");
    pc.printf("\n\rVal min : %f, Val max : %f", min_gaz, max_gaz);
}
else{
    gaz_calibre = false;
    pc.printf("\n\r-----ERROR : reessayez-----");
}
```

Figure 19 : Calibrage

e. Fichier d'enregistrement

Notamment pour les données minimum et maximum de la poignée de gaz, il serait judicieux de les enregistrer dans la mémoire du microcontrôleur pour ne pas avoir à refaire le calibrage à chaque démarrage. Pour cela on crée un fichier dans lequel on renseigne un certain nombre de données (le reste sera vu plus tard dans ce rapport) puis, à chaque démarrage on lit ce fichier et on assigne les valeurs à des variables. Pour réaliser l'écriture voilà comme cela se fait :

```

fp = fopen("/local/gr2_conf.txt", "w");
if(fp != NULL){
    fprintf(fp, "%f %f %d %d %d %f", min_gaz, max_gaz, v_bridage, cpt_hall_total, cpt_hall_partiel, capa_restante);
    fclose(fp);
}
else{
    pc.printf("\n\rERROR : Probleme avec le fichier");
}

```

Figure 20 : Enregistrement fichier

Et pour la lecture :

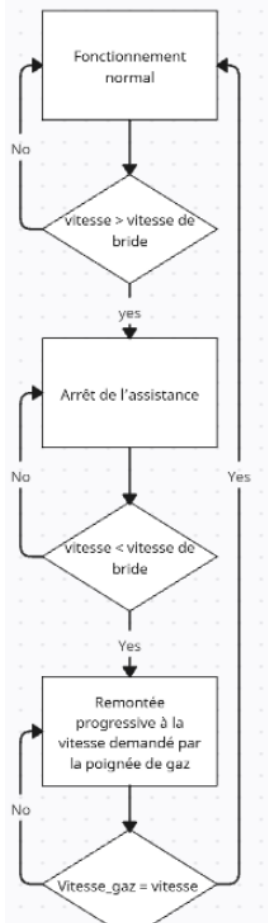
```

FILE *fp = fopen("/local/gr2_conf.txt", "r");
if(fp != NULL){
    fscanf(fp, "%f %f %d %d %d %f", &min_gaz, &max_gaz, &v_bridage, &cpt_hall_total, &cpt_hall_partiel, &capa_restante);
    //pc.printf("\n\rmin : %f, max : %f, v_bridage : %d, cpt_hall_total : %d, cpt_hall_partiel : %d, capacite restante : %f");
    fclose(fp);
    gaz_calibre = true;
}
else{
    min_gaz = 0.158;
    max_gaz = 0.374;
    v_bridage = 25;
    gaz_calibre = true;
    cpt_hall_total = 0;
    cpt_hall_partiel = 0;
}

```

Figure 21 : lecture fichier

f. Bridage et freinage



Nous voulons piloter un véhicule à assistance électrique, ici une roue de vélo, nous devons donc intégrer une bride. Nous allons également gérer le freinage. Pour ces 2 actions, le but va être de remonter la vitesse en douceur que ce soit après le frein ou une fois que la vitesse est descendue en dessous de la bride. De plus, pour cette dernière, cela évitera les à-coups à ladite vitesse. Pour cela on suit cet algorithme :

A noter que le frein reste prioritaire en tout point, et que lorsqu'il est enclenché, on arrête l'assistance et on passe à l'état « Remontée progressive de la vitesse ».

Le code de cet algorithme, qui s'effectue toutes les 5ms, se trouve ci-dessous :

```
void echantillonage_5ms(){
    readPLD = MyPLD.read();
    BRAKE = 0x01 & (readPLD >> 5);
    if(BRAKE == 0){
        etat_echantillonage_5ms = 10;
    }

    switch(etat_echantillonage_5ms){
        case 0 : if(vitesse_km_h >= v_bridage) etat_echantillonage_5ms = 21; break;
        case 10: if(BRAKE == 1) etat_echantillonage_5ms = 11; break;
        case 11: if(PWM_rampe >= DC_gaz) etat_echantillonage_5ms = 0; break;
        case 21 : if(vitesse_km_h < v_bridage) etat_echantillonage_5ms = 11; break; // retour en fonctionnement normal
    }

    switch(etat_echantillonage_5ms){
        case 0: //fonctionnement normal
            if(mode_auto){
                MyPLD.write(DC_gaz);
            }
            else{
                MyPLD.write(duty_cycle_manu);
            }
            break;
        case 10 : //arret de la roue lors du frein
            MyPLD.write(0);
            PWM_rampe = 0;
            break;
        case 11 : //augmentation de la vitesse en douceur
            PWM_rampe++;
            MyPLD.write(PWM_rampe);
            break;
        case 21 : //bridage
            MyPLD.write(0);
            PWM_rampe = 0;
            break;
    }
}
```

Figure 22 : Algorithme remontée progressive

g. Compteurs kilométriques

A présent, nous allons rajouter 2 compteurs kilométriques. Un compteur total qui ne pourra pas être remis à zéro et un partiel qui pourra être remis à zéro via la console.

Pour cela il suffit de modifier un peu la formule de la vitesse afin d'obtenir ces formules :

```
cpt_km_all = ((cpt_hall_total*2*PI*RADIUS)/(6*NBPOLES)) / 1000;
cpt_km_partiel = ((cpt_hall_partiel*2*PI*RADIUS)/(6*NBPOLES)) / 1000;
```

Figure 23 : compteur kilométrique

Et on ajoute une option dans le menu pour remettre à zéro le compteur partiel.

h. Capteurs

Dans le système, nous avons 3 capteurs : un capteur de courant instantané, un capteur de tension et un capteur de température. Le but ici va être de réunir toutes ces informations afin de pouvoir les afficher.

Pour le capteur de tension, il suffit de faire une fonction affine avec 2 points du capteur, afin de déterminer sa caractéristique.

```
var_Vbat = Vbat.read();  
var_Vbat = (var_Vbat*25.0)/0.326252;
```

Figure 24 : capteur batterie

Pour la température, on utilise un capteur LM335, avec une résolution de 10mV/°K. Pour ce faire, on récupère la valeur analogique du capteur que l'on multiplie par un Vref, ici 3V3 afin d'avoir la valeur en tension. On fait ensuite les calculs pour avoir la température en Kelvin puis en degrés Celsius, comme visible ci-dessous :

```
var_Vtemp = Vtemp.read();  
tension = var_Vtemp * V_REF;  
temperature_K = tension / 0.01;  
temperature_C = temperature_K - 273.15;
```

Figure 25 : capteur température

Enfin, pour le capteur de courant, c'est un petit peu plus délicat. Le capteur a un gain, ici 0.185 V/A, mais la tension passe également dans un pont diviseur, qu'il faut donc prendre en compte. Ci-dessous l'ensemble du calcul pour trouver la valeur instantanée du courant :

```
var_VIdc = VIdc.read();  
var_VIdc = (var_VIdc * V_REF * (1/val_pont_div) - 2.5) / G_Ic_capt;
```

Figure 26 : capteur courant

Avec ce courant instantané, il nous est donc possible de mettre à jour la capacité restante dans la batterie du vélo. Pour cela il faut soustraire la capacité restante au courant consommé tous les x temps, ici toutes les 100ms. On a donc ceci :

```
capa_restante = capa_restante - (var_VIdc * 1000) * DELTA_T;  
pourcent_batterie = (capa_restante / Capa_init) * 100;  
if (pourcent_batterie <= 0){  
    pourcent_batterie = 0;  
    capa_restante = 0;  
}  
else if (pourcent_batterie >= 100){  
    pourcent_batterie = 100.0;  
    capa_restante = Capa_init;  
}
```

Figure 27 : capacité batterie

i. Liaison avec le serveur Web

L'objectif est ensuite de créer une connexion avec le web afin de faire transiter des données vers l'extérieur et les afficher ensuite dans une application Web voir même sur un téléphone.

Pour cela, nous avons une bibliothèque qui est prévu pour. Il nous suffit de rentrer une adresse IP qui correspond à notre table de TP (pour avoir une adresse unique) et le microcontrôleur se comporte comme serveur, et nous pouvons y avoir accès depuis n'importe quel navigateur.

```
eth.init("134.59.29.89", "255.255.255.192", "134.59.29.126");
```

Figure 28 : adresse IP serveur mbed

Dans cette bibliothèque, nous avons une méthode du nom de *Send Json(char)* qui sert à envoyer une chaine JSON sur le Web. Pour que cette fonction soit appelée, on met en place une interruption lorsque le serveur (donc le microcontrôleur) reçoit une requête de l'extérieur. Cette interruption appelle une fonction qui va créer la chaine JSON et l'envoyer vers l'extérieur.

```
void CGI_Function(void){ // cgi function that patch web data to empty web page
    char ma_chaine[200]={}; // needed to form html response
    sprintf(ma_chaine, "{\"vitesse\" : %d, \"Batterie\" : %d, \"Temperature\" : %d, \"kilometre_par\" : %d}\n", speed, battery, temperature, kmh);
    SendJson(ma_chaine);
    giCounter=giCounter+1;
}
```

Figure 29 : chaine JSON

Nous pouvons donc à présent récupérer cette chaine JSON sur des outils tels que NodeRED pour pouvoir les interpréter et en faire une interface graphique, et c'est ce que nous allons voir à présent.

IV. NodeRED

Une fois la chaîne JSON envoyée sur le WEB nous pouvons, pour commencer vérifier qu'elle est belle et bien envoyée. Pour cela, il suffit d'ouvrir un navigateur et d'entrer l'adresse préalablement choisie, en l'occurrence pour nous, il s'agit de "134.59.29.89". Une fois cela fait, on devrait avoir une page qui s'ouvre avec la chaîne de caractères envoyée.

Ensuite, nous pouvons lancer notre serveur Node Red local, comme nous l'avons appris durant la SAE LORA. Il suffit alors d'aller à l'adresse du serveur, soit "127.0.0.1/1880/".

La première étape dans le Node Red, est de faire une requête http à notre microcontrôleur afin de récupérer la chaîne de caractères.

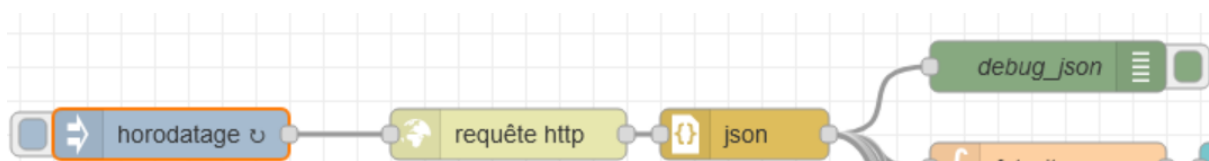


Figure 30 : Nodered : requête

Le bloc "horodatage" permet de déclencher le programme Node Red, le Bloc "requête http" est fourni par Node Red, il suffit alors d'entrer l'adresse de l'envoi de la chaîne JSON.

Méthode	GET
URL	http://134.59.29.89/
Charge utile	Ignorer
☐ Activer la connexion sécurisée (SSL/TLS)	
☐ Utiliser l'authentification	
☐ Activer le maintien de la connexion	
☐ Utiliser un proxy	
☐ N'envoyer que des réponses non-2xx au noeud Catch	
☒ Désactiver l'analyse HTTP stricte	

Figure 31 : Paramétrage requête

Il faut ensuite, comme vu sur la figure précédente, ajouter le bloc "JSON", ainsi que le bloc "debug", qui nous permettent de voir si la chaîne de caractères est envoyée et de

recupérer le chemin des différentes variables. Pour cela il faut aller dans l'onglet du debug et copier le chemin.



Figure 32 : chemin de la variable

Ensuite nous pouvons alors chercher la valeur de chaque variable.

```
1 msg.payload = msg.payload.vitesse;  
2 return msg;
```

On attribue à msg la valeur qui est envoyée au chemin de la vitesse par exemple.

Il faut ensuite le faire pour toutes les variables. On peut alors y ajouter des gauges afin de pouvoir afficher toutes nos variables sur un dashboard.

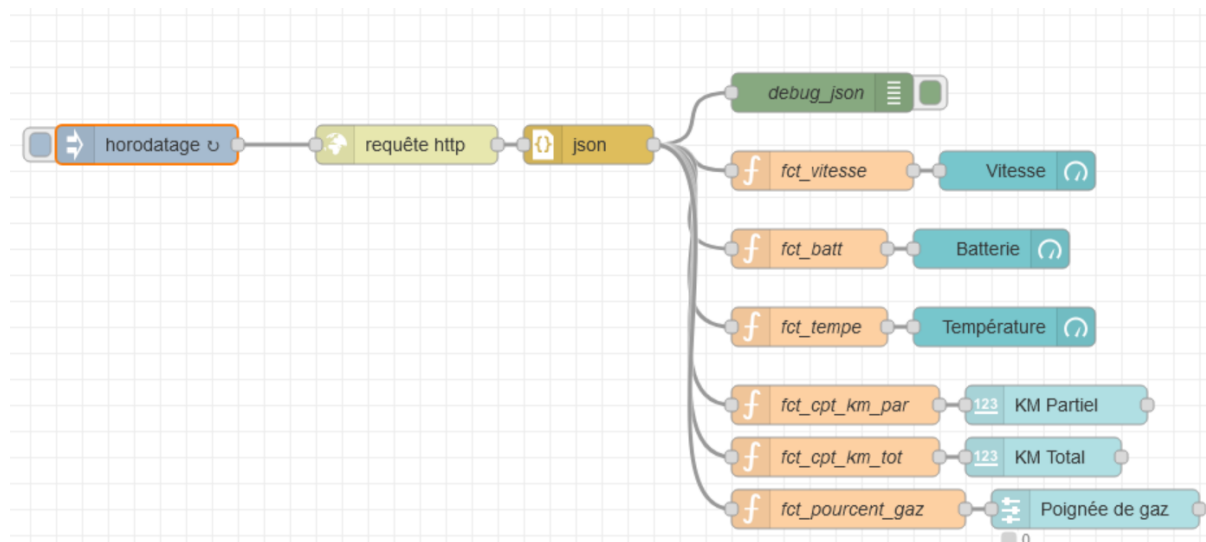


Figure 33 : nodered complet

Figure 34 : menu gauge

Dans le bloc du “Gauge”, nous pourrions régler l’intervalle, le type de design, la position du Gauge, son titre et son unité.

Une fois cela fait pour toutes les variables, on peut obtenir un dashboard de ce type.

La fabuleuse roue (et pas roux!!)

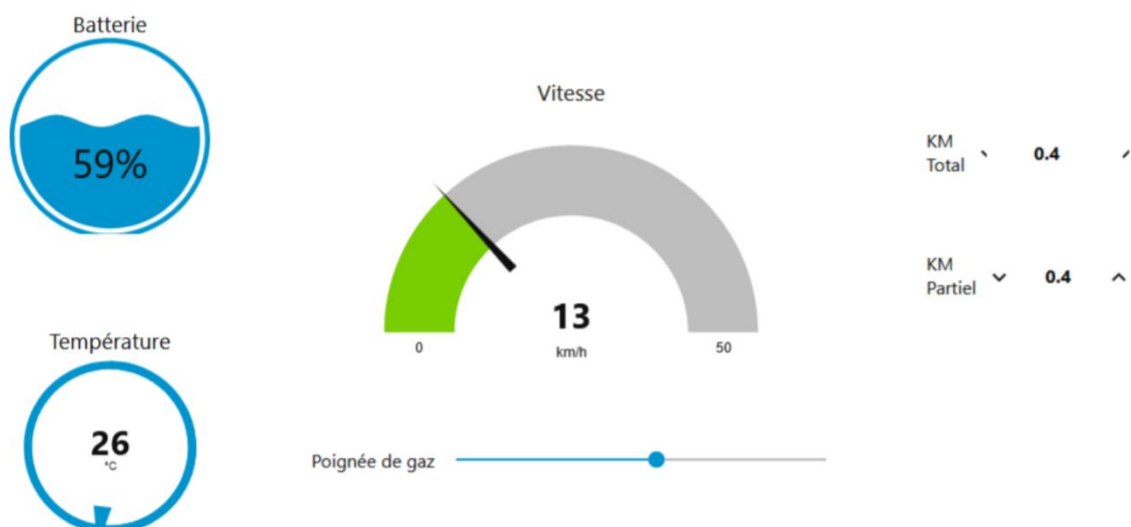


Figure 35 : Dashboard

On y retrouve les différentes variables envoyées depuis le MBED.

Conclusion

Ce projet a été intéressant et nous avons appris des notions supplémentaires en électronique numérique et donc en VHDL. Nous avons également pu réutiliser des compétences acquises préalablement durant les 4 semestres de la formation à l'IUT.

Il y a eu un grand nombre de tâches et d'objectifs à atteindre, ce que nous avons très bien réussi à nous deux, prenant même de l'avance sur les séances. Nous sommes donc satisfaits de notre réussite dans cette SAE qui nous apporté de nouvelles connaissances.

Ces connaissances nous seront très certainement utiles pour nos parcours, tant d'études que professionnels futurs.

Table des figures

Figure 1 : VAE	4
Figure 2 : Synoptique du projet	5
Figure 3 : Onduleur	6
Figure 4 : Position 1 et 2	6
Figure 5 : Position 3 et 4	7
Figure 6 : Position 5 et 6	7
Figure 7 : VHDL : PWM	8
Figure 8 : Bloc PWM	8
Figure 9 : VHDL filtre HALL	9
Figure 10 : bloc filtre HALL	10
Figure 11 : VHDL Collecteur Numérique	10
Figure 12 : BDF entier	11
Figure 13 : IHM1	12
Figure 14 : menu rapport cyclique	12
Figure 15 : Masquage du MyPLD.read()	13
Figure 16 : Affichage des données du PLD	14
Figure 17 : Calcul vitesse	14
Figure 18 : Calcul poignée de gaz	15
Figure 19 : Calibrage	15
Figure 20 : Enregistrement fichier	16
Figure 21 : lecture fichier	16
Figure 22 : Algorithme remontée progressive	17
Figure 23 : compteur kilométrique	17
Figure 24 : capteur batterie	18
Figure 25 : capteur température	18
Figure 26 : capteur courant	18
Figure 27 : capacité batterie	18
Figure 28 : adresse IP serveur mbed	19
Figure 29 : chaine JSON	19
Figure 30 : Nodered : requête	20
Figure 31 : Paramétrage requête	20
Figure 32 : chemin de la variable	21
Figure 33 : nodered complet	21
Figure 34 : menu gauge	22
Figure 35 : Dashboard	22